Les cinq choses pour lire n’importe lequel de ces scripts

Lundi 19 octobre · 45 minutes · à garder sous les yeux toute la semaine

> Vous n’allez pas apprendre Python cette semaine. Vous allez apprendre à **lire** cinq formes qui reviennent dans tous les scripts, et à en modifier une à la fois.
>
> Ces cinq choses suffisent à comprendre chaque ligne de chaque script de la semaine. Il n’y en a pas une sixième.

---

1 · Une variable, c’est une étiquette posée sur une valeur

```
mot_de_passe = "secret"
nombre_de_lignes = 2000
```

Le signe `=` ne veut pas dire « égale ». Il veut dire : **« range cette valeur, et appelle-la comme ça »**.

On lit de droite à gauche : *la valeur 2000 est rangée sous le nom* `nombre_de_lignes`.

Après, écrire `nombre_de_lignes` revient à écrire `2000`.

> **À vous.** Dans `lire_la_base.py`, trouvez trois variables. Qu’y a-t-il rangé dans chacune ?

---

2 · Appeler une fonction : `nom(ce_qu_on_lui_donne)`

```
print("bonjour")
```

`print` est une fonction : elle fait quelque chose. Les parenthèses contiennent **ce qu'on lui donne pour travailler**.

```
print(nombre_de_lignes)     # affiche 2000
```

S'il y a plusieurs choses à donner, on les sépare par une virgule :

```
pd.read_csv("export.csv", sep=";")
```

> Ici on donne deux choses : le nom du fichier, et le fait que les colonnes sont séparées par des points-virgules.
>
> **`sep=";"` se lit : « le réglage *sep* vaut point-virgule ».** Beaucoup de fonctions ont des réglages comme celui-là. On ne les apprend pas par cœur : on les cherche quand on en a besoin.

> **À vous.** Dans `lire_la_base.py`, trouvez l'appel à `to_csv`. Combien de réglages lui donne-t-on, et à quoi sert chacun ? *(Ils sont commentés juste au-dessus.)*

---

3 · Une méthode : `objet.action()` — l'objet d'abord, l'action ensuite

C'est la forme la plus fréquente, et la plus déroutante au début.

```
tableau.head(5)
```

Ça se lit : « **le tableau, donne-moi tes 5 premières lignes** ». L'objet est à gauche du point, ce qu'on lui demande est à droite.

C'est la différence avec une fonction :

`print(tableau)`	on **donne** le tableau à `print`

`tableau.head()`	on **demande** quelque chose au tableau

Et on peut enchaîner, de gauche à droite :

```
tableau["canal_reservation"].str.strip()
```

« Dans le tableau, prends la colonne canal_reservation ; traite-la comme du texte ; enlève les espaces au début et à la fin. »

> **À vous.** Dans `profiler.py`, trouvez trois méthodes. Que demande-t-on, et à qui ?

---

4 · Le texte et le nombre ne sont pas la même chose

```
"150"      ce sont trois caractères : un 1, un 5, un 0
150        c'est un nombre
```

Ça paraît un détail. **C’est la source d’erreur numéro un de la semaine.**

```
"150" + "30"   donne "15030"    les textes se collent bout à bout
 150  +  30    donne  180        les nombres s'additionnent
```

Python ne vous préviendra pas. Il fera ce que vous lui avez demandé, et le résultat sera faux.

> **Jeudi matin, cette ligne vous attend.** Quand vous y serez, revenez ici.

---

5 · Lire une erreur Python : la dernière ligne d’abord

Quand un script s’arrête, Python affiche un long bloc rouge. **Vous n’avez pas à le lire en entier.**

```
Traceback (most recent call last):
  File "traiter_export.py", line 38, in <module>
    ...
KeyError: 'canal_reservation'
```

Trois réflexes, dans cet ordre :

1. **La dernière ligne** dit le type d'erreur et ce qui pose problème : `KeyError: 'canal_reservation'`.
2. **La ligne avec le nom de VOTRE fichier** dit où ça s'est produit : `line 38`. Ignorez les lignes qui parlent de fichiers que vous n'avez pas écrits.
3. **Le nom entre guillemets** est l'indice : c'est `canal_reservation` qui pose problème. Pas autre chose.

> Vous avez déjà fait exactement ça le 22 septembre, sur un `KeyError: 'nom'`. **C'est la même erreur, et la même méthode.** Ressortez votre mémo de lecture d'erreur.

Ce qui n'est PAS dans cette fiche, et c'est voulu

Vous n'avez pas besoin cette semaine de savoir écrire une boucle, une condition ou une fonction. Vous avez besoin de **reconnaître** ces cinq formes quand vous les voyez, et de modifier ce qu'il y a dedans.

Écrire viendra. Lire d'abord.